

P4006R0

Transparent Function Objects for Shift Operators

Draft Proposal, 2026-02-03

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes adding `std::bit_lshift` and `std::bit_rshift` transparent function objects to complete the set of bitwise operation functors in `<functional>`. The original N3421 proposal acknowledged shift operators could be useful but deferred them as "slightly beyond completely trivial to specify." With the transparent functor pattern now well-established, we address these deferred operators and comprehensively document why other operators should not be added at the same time.

Table of Contents

1 Introduction

- 1.1 Historical Context from N3421
- 1.2 Purpose of This Paper

2 Motivation

- 2.1 Current State
- 2.2 Relationship to P3793R1
- 2.3 Use Cases

- 2.3.1 Completing the Bitwise Operator Family
- 2.3.2 Algorithm Composition
- 2.3.3 Generic Programming
- 2.3.4 Supporting Generic Library Customization

3 Design Rationale

- 3.1 Other Operators Not Proposed
- 3.2 Operator Overload Context

4 Implementation Experience

5 Impact on Existing Code

6 Teachability

7 Proposed Design

8 Design Alternatives

- 8.1 Naming
 - 8.1.1 Chosen Names: `bit_lshift` and `bit_rshift`
 - 8.1.2 Alternative names

9 Wording

10 Appendix: Operator Coverage Summary

References

Informative References

§ 1. Introduction

Since C++14, the standard library has provided transparent function objects (functors with `void` template specialization and `is_transparent` member type) for most C++ operators, introduced by [N3421]. However, the shift operators (`<<` and `>>`) were not included.

§ 1.1. Historical Context from N3421

The original [N3421] proposal noted that shift operators "could be useful" but deferred them as "slightly beyond completely trivial to specify." The author raised questions about design details for operators like address-of (should it use `operator&` or `std::addressof()`?), which warranted deferral. However, shift operators have no such ambiguity—they simply forward to `<<` and `>>` following the same transparent pattern as `bit_and<>`, `plus<>`, etc.

§ 1.2. Purpose of This Paper

This paper proposes adding `std::bit_lshift<>` and `std::bit_rshift<>` to complete the set of bitwise operator function objects in `<functional>`. The design is straightforward—these functors follow the same transparent pattern as existing operators like `bit_and<>` and `plus<>`, simply forwarding to the built-in `<<` and `>>` operators.

§ 2. Motivation

§ 2.1. Current State

The standard library currently provides transparent function objects for:

- **Bitwise operations:** `bit_and<>`, `bit_or<>`, `bit_xor<>`, `bit_not<>`
- **Arithmetic operations:** `plus<>`, `minus<>`, `multiplies<>`, `divides<>`, `modulus<>`, `negate<>`
- **Comparison operations:** `equal_to<>`, `not_equal_to<>`, `greater<>`, `less<>`, `greater_equal<>`, `less_equal<>`
- **Logical operations:** `logical_and<>`, `logical_or<>`, `logical_not<>`

Notably absent are transparent function objects (operator wrappers) for the bitwise shift operators `<<` and `>>`.

§ 2.2. Relationship to P3793R1

This proposal is complementary to [P3793R1], which proposes `std::shl` and `std::shr` as direct function calls in `<bit>` (similar to `std::rotr/std::rotr`).

- **P3793R1** provides named functions in `<bit>` that define behavior for shift operations that would otherwise be undefined (e.g., shifting by negative amounts or amounts $\geq$ width)
- **P4006 (this paper)** provides transparent function objects in `<functional>` (like `bit_and<>`, `bit_or<>`) that forward directly to the built-in `<<` and `>>` operators, preserving all their semantics including undefined behavior

The facilities serve different purposes:

- `std::shl(x, n)` provides defined behavior even for edge cases
- `std::bit_lshift<>{}(x, n)` is a transparent wrapper for `x << n` with identical semantics to `<<`.

§ 2.3. Use Cases

§ 2.3.1. Completing the Bitwise Operator Family

The standard library provides transparent function objects for all other bitwise operators (`bit_and<>`, `bit_or<>`, `bit_xor<>`, `bit_not<>`). Omitting shift operators creates an inconsistency—users must write verbose lambdas for shifts while using concise functors for other bitwise operations. This proposal completes the family for consistency and uniformity.

§ 2.3.2. Algorithm Composition

Transparent function objects enable heterogeneous comparisons and operations in algorithms:

```
// Without P4006 – verbose lambda
std::transform(values.begin(), values.end(),
               shifts.begin(),
               results.begin(),
               [](auto v, auto s) { return v << s; });

// With P4006 – concise and self-documenting
std::transform(values.begin(), values.end(),
               shifts.begin(),
               results.begin(),
               std::bit_lshift<>{});
```

§ 2.3.3. Generic Programming

Transparent function objects exist to enable uniform passing of operators to generic algorithms and functions. Without `std::bit_lshift<>` and `std::bit_rshift<>`, there is a gap in this uniformity since shift operators cannot be passed the same way as other binary operators like `std::plus<>` or `std::bit_and<>`.

Example:

```
// Library code that needs to work with any binary operator
template<typename BinOp>
auto apply_operation(auto lhs, auto rhs, BinOp op) {
    return op(lhs, rhs);
}

// Without proposal: can't pass shift operators uniformly
apply_operation(x, y, std::plus<>{});           // ✓
apply_operation(x, y, std::bit_and<>{});        // ✓
apply_operation(x, y, [](auto a, auto b) { return a << b; }); // ✗ verbose

// With proposal:
apply_operation(x, y, std::bit_lshift<>{});     // ✓
```

§ 2.3.4. Supporting Generic Library Customization

Generic libraries benefit from uniform operator discovery through transparent function objects. For example, work on `std::simd` support for User-Defined Types (UDTs) in [P2964R1] explores customization points for optimizing operations on specific element types. While this proposal is independent of P2964R1's acceptance, it enables such libraries to handle all operators uniformly when providing customization points.

For operations like addition, implementations can check that `std::plus<>` is valid for the element type. Without `std::bit_lshift<>` and `std::bit_rshift<>`, shift operators lack this uniform discovery mechanism, creating an asymmetry in customization point design.

§ 3. Design Rationale

§ 3.1. Other Operators Not Proposed

This proposal focuses solely on shift operators because they are the only operators that:

1. Follow the same pure-function pattern as existing transparent functors
2. Complete the bitwise operator family (`bit_and`, `bit_or`, `bit_xor`, `bit_not` → `bit_lshift`, `bit_rshift`)
3. Have no semantic ambiguities

Other operators remain unsuitable for reasons detailed in [§ 10 Appendix: Operator Coverage Summary](#).

§ 3.2. Operator Overload Context

While `operator<<` and `operator>>` are commonly overloaded for stream I/O in C++, the `bit_` prefix clearly indicates these functors wrap the bitwise shift operations, not stream operations. This naming convention follows the existing pattern where `bit_and<>` wraps `operator&` (not `std::addressof`) and `bit_or<>` wraps `operator|`.

These functors work with any type that provides `operator<<` or `operator>>`, including:

- Built-in integral types performing bitwise shifts
- User-defined types with overloaded shift operators (whether for bitwise shifts or other purposes)

The functors are completely transparent—they forward to whatever `operator<</operator>>` means for the operand types.

§ 4. Implementation Experience

This proposal follows the exact same pattern as existing transparent function objects in `<functional>`. The implementation is trivial (as N3421 predicted):

```
template<> struct bit_lshift<void> {
    template<class T, class U>
    constexpr auto operator()(T&& lhs, U&& rhs) const
        noexcept(noexcept(std::forward<T>(lhs) << std::forward<U>(rhs)))
```

```

-> decltype(std::forward<T>(lhs) << std::forward<U>(rhs))
{
    return std::forward<T>(lhs) << std::forward<U>(rhs);
}

using is_transparent = void;
};

```

The author has prototyped this implementation and tested it with the use cases in [§2.3 Use Cases](#), confirming it works as expected with no surprises.

§ 5. Impact on Existing Code

None. This is a pure library addition with no changes to existing facilities. Code using lambdas for shift operations continues to work unchanged.

§ 6. Teachability

Users already understand transparent function objects. Teaching materials simply add:

- "Use `std::bit_lshift<>{}>` to pass the left-shift operator to algorithms"
- "Use `std::bit_rshift<>{}>` to pass the right-shift operator to algorithms"

This follows the same pattern as teaching `std::plus<>{}>`, `std::bit_and<>{}>`, etc. The `bit_` prefix makes the purpose immediately clear.

§ 7. Proposed Design

Add two new transparent function objects to `<functional>`:

```

template<class T = void>
struct bit_lshift {
    constexpr T operator()(const T& lhs, const T& rhs) const {
        return lhs << rhs;
    }
};

```

```

template<class T = void>
struct bit_rshift {
    constexpr T operator()(const T& lhs, const T& rhs) const {
        return lhs >> rhs;
    }
};

// Transparent specializations
template<>
struct bit_lshift<void> {
    template<class T, class U>
    constexpr auto operator()(T&& lhs, U&& rhs) const
        noexcept(noexcept(std::forward<T>(lhs) << std::forward<U>(rhs)))
        -> decltype(std::forward<T>(lhs) << std::forward<U>(rhs)) {
        return std::forward<T>(lhs) << std::forward<U>(rhs);
    }

    using is_transparent = void;
};

template<>
struct bit_rshift<void> {
    template<class T, class U>
    constexpr auto operator()(T&& lhs, U&& rhs) const
        noexcept(noexcept(std::forward<T>(lhs) >> std::forward<U>(rhs)))
        -> decltype(std::forward<T>(lhs) >> std::forward<U>(rhs)) {
        return std::forward<T>(lhs) >> std::forward<U>(rhs);
    }

    using is_transparent = void;
};

```

§ 8. Design Alternatives

§ 8.1. Naming

§ 8.1.1. Chosen Names: `bit_lshift` and `bit_rshift`

This proposal uses `bit_lshift<>` and `bit_rshift<>` for the following reasons:

1. **Consistency with existing bitwise operators:** The standard library already uses the `bit_` prefix for bitwise operation functors: `bit_and<>`, `bit_or<>`, `bit_xor<>` and `bit_not<>`. Using `bit_lshift<>` and `bit_rshift<>` completes this family.
2. **Avoids naming conflict:** C++20 added `std::shift_left` and `std::shift_right` as algorithms in `<algorithm>` ([`alg.shift`]) that shift ranges of elements left/right. Using `bit_lshift/bit_rshift` avoids this conflict.
3. **Clear semantic distinction from P3793R1:** [P3793R1] proposes `std::shl()` and `std::shr()` functions in `<bit>` that provide defined behavior for shift operations that would otherwise be undefined. In contrast, `bit_lshift<>` and `bit_rshift<>` are transparent wrappers that forward directly to the built-in `<<` and `>>` operators, preserving all their semantics including undefined behavior.

§ 8.1.2. Alternative names

Using `shl<>` and `shr<>` was considered for brevity and potential alignment with [P3793R1]. However:

- **Semantic mismatch:** The P3793R1 functions define behavior for cases where `<</>>` have undefined behavior. These functors simply forward to `<</>>` without any special handling.
- **Inconsistency:** Would break the `bit_*` naming pattern for bitwise operations.

The names `shift_left<>` and `shift_right<>` would be more descriptive, but these names are already used for range-shifting algorithms in `<algorithm>` since C++20.

Using `lshift<>` and `rshift<>` without the `bit_` prefix was considered but the `bit_` prefix immediately identifies these as bitwise operations, distinguishing them from other potential shift operations.

§ 9. Wording

Add to [version.syn]:

```
#define __cpp_lib_bitwise_shift_functors YYYYMMML // also in <functional>
```

Add to **[functional.syn]** in `<functional>`:

```
namespace std {  
    // ...  
  
    // [bitwise.operations], bitwise operations  
    template<class T = void> struct bit_and;  
    template<class T = void> struct bit_or;  
    template<class T = void> struct bit_xor;  
    template<class T = void> struct bit_not;  
    template<class T = void> struct bit_lshift;  
    template<class T = void> struct bit_rshift;  
  
    // ...  
}
```

Add new subsection `[bitwise.operations.shift]` after `[bitwise.operations]`:

```
template<class T = void> struct bit_lshift {
    constexpr T operator()(const T& lhs, const T& rhs) const;
};
```

Effects: Equivalent to: `return lhs << rhs;`

```
template<class T = void> struct bit_rshift {
    constexpr T operator()(const T& lhs, const T& rhs) const;
};
```

Effects: Equivalent to: `return lhs >> rhs;`

```
template<> struct bit_lshift<void> {
    template<class T, class U>
        constexpr auto operator()(T&& lhs, U&& rhs) const
            noexcept(noexcept(std::forward<T>(lhs) << std::forward<U>(rhs)))
            -> decltype(std::forward<T>(lhs) << std::forward<U>(rhs));

    using is_transparent = unspecified;
};
```

Effects: Equivalent to: `return std::forward<T>(lhs) << std::forward<U>(rhs);`

```
template<> struct bit_rshift<void> {
    template<class T, class U>
        constexpr auto operator()(T&& lhs, U&& rhs) const
            noexcept(noexcept(std::forward<T>(lhs) >> std::forward<U>(rhs)))
            -> decltype(std::forward<T>(lhs) >> std::forward<U>(rhs));

    using is_transparent = unspecified;
};
```

Effects: Equivalent to: `return std::forward<T>(lhs) >> std::forward<U>(rhs);`

[Note 1: `bit_lshift` and `bit_rshift` forward directly to the built-in operators and preserve all their semantics, including undefined behavior when the shift amount is negative or exceeds the width of the promoted left operand. — end note]

[Note 2: While `operator<<` and `operator>>` are commonly overloaded for stream I/O, the `bit_` prefix clearly indicates these functors wrap the bitwise shift operations, not stream opera-

tions. The functors work with any types that provide these operators. — *end note*]

§ 10. Appendix: Operator Coverage Summary

This appendix documents which C++ operators have transparent function objects and why others are not proposed.

Category	Operators	Status
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , unary <code>-</code>	✓ Covered (<code>plus<></code> , <code>minus<></code> , <code>multiplies<></code> , <code>divides<></code> , <code>modulus<></code> , <code>negate<></code>)
Bitwise (non-shift)	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code>	✓ Covered (<code>bit_and<></code> , <code>bit_or<></code> , <code>bit_xor<></code> , <code>bit_not<></code>)
Bitwise (shift)	<code><<</code> , <code>>></code>	⊕ This proposal (<code>bit_lshift<></code> , <code>bit_rshift<></code>)
Comparison	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	✓ Covered (<code>equal_to<></code> , <code>not_equal_to<></code> , etc.)
Logical	<code>&&</code> , <code> </code> , <code>!</code>	✓ Covered (<code>logical_and<></code> , <code>logical_or<></code> , <code>logical_not<></code>)
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , etc.	✗ Mutating, incompatible with perfect forwarding
Increment/Decrement	<code>++</code> , <code>--</code>	✗ Mutating, prefix/postfix ambiguity
Member access	<code>-></code> , <code>.*</code> , <code>->*</code> , <code>[]</code>	✗ Context-dependent semantics
Other	<code>&</code> (address), <code>*</code> (deref), <code>,</code> , <code>()</code> , <code>new</code> , <code>delete</code> , unary <code>+</code>	✗ Various design issues (see N3421)

Why shift operators are different: Unlike excluded operators, shifts are pure functions with no side effects, no design ambiguities (unlike address-of's `operator&` vs `std::addressof()` question),

and complete the bitwise operator family.

§ References

§ Informative References

[N3421]

Stephan T. Lavavej. *Making Operator Functors greater<>*. 20 September 2012. URL: <https://wg21.link/n3421>

[P2964R1]

Daniel Towner, Ruslan Arutyunyan. *Allowing user-defined types in std::simd*. 22 May 2024. URL: <https://wg21.link/p2964r1>